



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
PRÓ-REITORIA DE GRADUAÇÃO
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIA
CURSO DE GRADUAÇÃO EM CURSO

Pedro Hércules Souza Grangeiro

**Inteligência Artificial Aplicada ao Texas Hold'em: Uma Abordagem com Aprendizado
por Reforço Profundo e Algoritmo de Predição**

PAU DOS FERROS

2026

Pedro Hércules Souza Grangeiro

**Inteligência Artificial Aplicada ao Texas Hold'em: Uma Abordagem com Aprendizado
por Reforço Profundo e Algoritmo de Predição**

Monografia apresentada à Universidade Federal
Rural do Semi-Árido como requisito parcial
para obtenção do título de Bacharel em Curso.

Orientador: Prof. Dr. Kennedy Reurison
Lopes - UFERSA

©Todos os direitos estão reservados à Universidade Federal Rural do Semi-Árido. O conteúdo desta obra é de inteira responsabilidade do (a) autor (a), sendo o mesmo, passível de sanções administrativas ou penais, caso sejam infringidas as leis que regulamentam a Propriedade Intelectual, respectivamente, Patentes: Lei nº 9.279/1996, e Direitos Autorais: Lei nº 9.610/1998. O conteúdo desta obra tornar-se-á de domínio público após a data de defesa e homologação da sua respectiva ata, exceto as pesquisas que estejam vinculadas ao processo de patenteamento. Esta investigação será base literária para novas pesquisas, desde que a obra e seu (a) respectivo (a) autor (a) seja devidamente citado e mencionado os seus créditos bibliográficos.

Ficha catalográfica elaborada pelo Sistema de Bibliotecas
da Universidade Federal Rural do Semi-Árido, com os dados fornecidos pelo(a) autor(a)

ATENÇÃO: Para confecção da sua Ficha Catalográfica, acesse o Gerador Automático através do SIGAA>Biblioteca>Serviços ao usuário>Solicitar ficha catalográfica.

Bibliotecária XXXXXXXX

O serviço de Geração Automática de Ficha Catalográfica para Trabalhos de Conclusão de Curso (TCC's) foi desenvolvido pelo Instituto de Ciências Matemáticas e de Computação da Universidade de São Paulo (USP) e gentilmente cedido para o Sistema de Bibliotecas da Universidade Federal Rural do Semi-Árido (SISBI-UFERSA), sendo customizado pela Superintendência de Tecnologia da Informação e Comunicação (SUTIC) sob orientação dos bibliotecários da instituição para ser adaptado às necessidades dos alunos dos Cursos de Graduação e Programas de Pós-Graduação da Universidade.

Pedro Hércules Souza Grangeiro

**Inteligência Artificial Aplicada ao Texas Hold'em: Uma Abordagem com Aprendizado
por Reforço Profundo e Algoritmo de Predição**

Monografia apresentada à Universidade Federal
Rural do Semi-Árido como requisito parcial
para obtenção do título de Bacharel em Curso.

Defendida em: _

BANCA EXAMINADORA

Kennedy Reurison Lopes, Prof. Dr. (UFERSA)
Presidente

Rosana Cibely Batista Rego, Profa. Dra. (UFERSA)
Membro Examinador

Marcos Daniel Nogueira Maia, Prof. Dr. (UFERSA)
Membro Examinador

Dedico este trabalho à toda minha família e meus amigos por terem me dado apoio e confiado em mim para produzir este trabalho.

AGRADECIMENTOS

Finalmente, após longos períodos tentando concluir este trabalho, noites de sono perdidas, abdicar de descanso, este trabalho está concluído, não posso deixar de agradecer as pessoas que sempre estiveram ao meu lado, me incentivando e me dando os sermões necessários, que sempre estavam lá quando eu pensei em desistir.

Primeiramente agradeço a minha família, principalmente minha mãe e meu pai que sempre perguntavam como estava a escrita e o desenvolvimento, minha mãe, que mesmo sem estar muito ligada a área se oferecia para pesquisar artigos e livros quando eu pedia, meu pai que foi quem me apresentou o Poker, que me mostrou que não era apenas um jogo de azar, mas sim um universo infinito de probabilidades e estatísticas, além de sempre me incentivarem a seguir a área dos meus sonhos, também agradeço meu padrasto e minha madrasta por me acolherem e sempre acreditarem no meu potencial, mesmo quando eu duvidava que eu ia conseguir eles estavam lá, perguntando o que meu programa fazia, como funcionava, me fazendo perceber que eu não estava tão perdido quanto eu pensava.

Agradeço meus amigos de mais longa data e que são como irmãos para mim, Felipe e Elivanaldo, sempre ao meu lado, me incentivando, falando o quanto era interessante e legal tudo o que eu fazia, me chamando para esfriar a cabeça quando eu estava prestes a enlouquecer, me chamando para jogar, para sair, até mesmo só para ficar jogando conversa fora e me fazendo esquecer a vontade de desistir.

Não posso deixar de agradecer meus amigos que fiz na UFERSA, Michelly, Mateus, Neilla, João Felype, Andrey, Angela, Pedro Emanuel, Davi, e claro, os mais próximos, que se tornaram apoios indispensáveis para mim em todo o meu tempo na faculdade, Pollyana e Pedro Vinícius, que são amigos que eu nunca posso perder, sempre me ajudando, me incentivando, brigando comigo e sempre me dizendo que eu era capaz, além de serem meus companheiros de jogatina, de estudos, de conversa, torço para que o destino nunca corte nosso contato, mesmo que nossos caminhos se dividam.

EPÍGRAFE

“Se você quer ser bem-sucedido, precisa ter dedicação total, buscar seu último limite e dar o melhor de si.”

(Ayrton Senna)

RESUMO

Este trabalho tem por objetivo o desenvolvimento de um agente inteligente utilizando redes neurais de aprendizado profundo para jogar o poquêr *Texas Hold'Em*. Tendo em vista que o jogo possui uma quantidade quase infinita de estados e acesso incompleto a informações, agentes por aprendizado tendem a cair em ótimos locais e ficarem estagnados sem evoluir. Este trabalho utilizou de ferramentas de *Deep Q-Network* (DQN) e algoritmos de predição, além de modelagem de recompensa e de um motor base 15 capaz de executar desempates de maneira robusta, usufruindo da *Application Programming Interface* (API) *TensorFlow.js* usando a ferramenta *Node.js* para desenvolvimento. Os gráficos gerados mostram o sucesso do agente no aprendizado, exibindo os locais em que o agente explorava e onde o mesmo explotava para receber o máximo possível de fichas. Tornando assim a exploração uma tática viável para agentes que jogam o *Texas Hold'Em*, já que eles conseguiam explorar as falhas dos rivais, também dispensando a necessidade de supercomputadores e mostrando que a linguagem *JavaScript* se mostrou capaz de desenvolver um agente de acordo com o que foi proposto.

Palavras-chave: Inteligência Artificial; Texas Hold'Em; Deep Q-Network; Modelagem de Recompensas; TensorFlow.js

ABSTRACT

This work aims to develop an intelligent agent using deep learning neural networks to play Texas Hold'Em Poker. Given that the game features a nearly infinite amount of states and incomplete access to information, learning agents tend to fall into local optima and stagnate without evolving. This study utilized Deep Q-Network tools and prediction algorithms, along with reward shaping and a Base 15 evaluation engine capable of robustly executing tie-breakers, leveraging the TensorFlow.js API and the Node.js environment for development. The generated graphs demonstrate the agent's success in learning, displaying the phases where the agent explored and where it exploited to acquire the maximum possible amount of chips. Thus, exploitation becomes a viable tactic for agents playing Texas Hold'Em, as they were able to exploit their rivals' flaws, simultaneously eliminating the need for supercomputers and demonstrating that the JavaScript language proved capable of developing an agent in accordance with the proposed objectives.

Keywords: Artificial Intelligence; Texas Hold'em; Deep Q-Network; Reward Shaping; TensorFlow.js

LISTA DE FIGURAS

Figura 1 – Arquitetura MVC	22
Figura 2 – Decaimento do fator de exploração <i>epsilon</i> (ϵ).	43
Figura 3 – Desempenho do agente ao longo das simulações	44

LISTA DE ABREVIATURAS E SIGLAS

API	<i>Application Programming Interface</i>
DQN	<i>Deep Q-Network</i>
EV	<i>Expected Value</i>
GPU	<i>Graphics Processing Unit</i>
HTML	<i>HyperText Markup Language</i>
HTTP	<i>Hypertext Transfer Protocol</i>
IA	Inteligência Artificial
JSON	<i>JavaScript Object Notation</i>
MCCFR	<i>Monte Carlo Counterfactual Regret Minimization</i>
MCTS	<i>Monte Carlo Tree Search</i>
MVC	<i>Model-View-Controller</i>
PARC	<i>Palo Alto Research Center</i>
PDF	<i>Portable Document Format</i>
ReLU	<i>Rectified Linear Unit</i>
RNA	Rede Neural Artificial
SIGAA	Sistema Integrado de Gestão de Atividades Acadêmicas
SISBI	Sistema de Bibliotecas da UFERSA
SUTIC	Superintendência de Tecnologia da Informação e Comunicação
TCC	Trabalho de Conclusão de Curso
UFERSA	Universidade Federal Rural do Semi-Árido
USP	Universidade de São Paulo

LISTA DE SÍMBOLOS

ε	Épsilon
γ	Gamma
θ	Theta
Σ	Somatório

SUMÁRIO

1	INTRODUÇÃO	14
2	FUNDAMENTAÇÃO TEÓRICA	16
2.1	O jogo pôquer <i>Texas Hold'Em</i>	16
2.1.1	A hierarquia das cartas	16
2.1.2	A configuração da modalidade	16
2.1.3	As combinações e o desempate	17
2.1.4	O desempate	19
2.1.5	As fases de aposta	19
2.1.6	As ações do jogador	20
2.1.7	Outros termos do pôquer	20
2.2	Estatística e o espaço de estados do <i>Texas Hold'Em</i>	21
2.3	Padrão <i>MVC</i>	21
2.4	Inteligência artificial e aprendizado por reforço	24
2.5	Método de <i>Monte Carlo</i> aplicado a uma rede neural jogadora de pôquer	25
2.5.1	Aplicação do método de <i>Monte Carlo</i> em jogos e pôquer	26
2.5.2	Integração do método de <i>Monte Carlo</i> com redes neurais para pôquer	27
2.5.2.1	Geração de rótulos a partir de simulações	28
2.5.2.2	Métodos de <i>Monte Carlo</i> em aprendizado por reforço	28
2.5.3	Aspectos práticos e considerações metodológicas	29
2.6	Redes neurais artificiais e <i>Q-Learning</i>	30
2.7	<i>Deep Q-Network</i>	31
2.8	Exploração vs. Exploração (<i>Epsilon-Greedy</i>)	32
2.9	Replay de Experiência (<i>Experience Replay</i>)	32
3	TRABALHOS RELACIONADOS	34
3.1	Abordagens baseadas em busca	34

3.2	Avanço em <i>Deep Reinforcement Learning</i>	34
3.3	Diferença arquitetural	35
4	METODOLOGIA	36
4.1	Tecnologias e Ferramentas Utilizadas	36
4.2	Arquitetura do ambiente e motor de regras	36
4.2.1	Estrutura <i>Model-View-Controller</i> (MVC)	36
4.2.2	Motor de avaliação - Base 15	37
4.3	Modelagem do agente inteligente (<i>DQN</i>)	38
4.3.1	Arquitetura e espaço de estados	38
4.3.2	Espaço de ações	40
4.3.3	Modelagem das recompensas	40
4.4	Cenários de simulação e Treinamento	41
4.4.1	O ambiente da mesa	41
4.4.2	A execução dos treinos	42
4.4.3	O decaimento do <i>Epsilon</i>	42
5	RESULTADOS	43
5.1	Validação do ambiente e do motor de regras	43
5.2	Evolução na taxa de exploração	43
5.3	Desempenho e lucro do agente	44
6	CONCLUSÕES E TRABALHOS FUTUROS	45
	REFERÊNCIAS	46
A	SAÍDA DO CONSOLE	48

1 INTRODUÇÃO

O poquêr, particularmente a modalidade *Texas Hold’Em*, é considerado um dos maiores desafios atuais para a inteligência artificial, pois envolve aspectos vitais como a tomada de decisão diante da incerteza, gerenciamento de risco e a necessidade de entender o comportamento dos oponentes. Ao contrário de jogos tradicionais como xadrez e *Go*, onde todas as informações estão disponíveis para os jogadores, o poquêr apresenta a característica de informação incompleta, exigindo decisões baseadas em dados parciais e sob a influência de elementos aleatórios. Essa particularidade tem incentivado o desenvolvimento de novas técnicas em *machine learning*, com foco especial em redes neurais e modelagem probabilística.

Neste cenário, os métodos que utilizam redes neurais artificiais têm se tornado cada vez mais significativos, sendo capazes de identificar padrões complexos a partir de extensas quantidades de dados, sem depender diretamente do conhecimento anterior do programador. O processo de treinamento desses modelos, que pode ser supervisionado, não supervisionado ou por reforço, possibilita a identificação de táticas de jogo, a previsão de jogadas dos adversários e a realização de escolhas otimizadas, fundamentais na mecânica do poquêr.

Contudo, a complexidade do espaço de estados no *Texas Hold’Em*, devido ao enorme número de combinações possíveis de cartas, apostas e estilos de jogo dos oponentes, torna impraticável a exploração de todas as opções disponíveis através de métodos convencionais. Para contornar esse desafio, destacam-se os algoritmos de *Monte Carlo*, incluindo o *Monte Carlo Tree Search* (MCTS) e o *Monte Carlo Counterfactual Regret Minimization* (MCCFR), que conseguem analisar amostras selecionadas das possibilidades do jogo, estimando chances de ganhar, retorno esperado e ajustando estratégias em tempo real. Essa metodologia simula diversos cenários possíveis, utilizando redes neurais para determinar os melhores caminhos, mesmo com limitações computacionais.

As mais recentes inovações neste campo envolvem a combinação de métodos de *Monte Carlo* com arquiteturas de redes neurais profundas, como as redes *actor-critic*, permitindo que o agente se desenvolva através de autojogo e levando a políticas que se aproximam do equilíbrio de *Nash*, um padrão de referência em jogos de soma zero. Exemplos como *DeepStack*, *Libratus* e *Pluribus* evidenciam a eficácia dessas ferramentas ao vencer campeões mundiais em competições reais, revelando táticas sólidas para partidas entre dois jogadores e também em ambientes com múltiplos competidores.

Dessa forma, a criação de uma rede neural capaz de jogar poquêr *Texas HoldEm* utili-

zando técnicas de *Monte Carlo* fornece uma contribuição significativa para o entendimento da IA aplicada a decisões complexas. Este estudo tem como objetivo executar modelagem teórica até práticas em ambientes simulados, demonstrando como a união entre aprendizado profundo e simulações probabilísticas pode resultar em agentes de alto desempenho em cenários com informações incompletas e vários adversários.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 O jogo pôquer *Texas Hold’Em*

O pôquer é um dos jogos mais populares da história, como dito por Barbosa, Freitas e Neves (2005), possuindo várias modalidades, sendo algumas como o pôquer *Head’s Up*, que consiste em um contra um direto, sendo a modalidade mais famosa a *Texas Hold’Em*, o pôquer é um jogo composto por muitos fatores, sendo os mais importantes para entendê-lo os seguintes:

2.1.1 A hierarquia das cartas

Diferentemente dos jogos que envolvem o baralho, no pôquer, o ás é a carta de maior valor, sendo a seguinte hierarquia das cartas:

1. Dois(2) 
2. Três(3) 
3. Quatro(4) 
4. Cinco(5) 
5. Seis(6) 
6. Sete(7) 
7. Oito(8) 
8. Nove(9) 
9. Dez(10) 
10. Valete(J) 
11. Dama(Q) 
12. Rei(K) 
13. Ás (A) 

sendo essa a ordem crescente dos valores das cartas do pôquer, é importante destacar que o naipe não influencia no fator de valor das cartas, sendo assim, um 8 de espadas e um 8 de paus têm o mesmo valor.

2.1.2 A configuração da modalidade

No *Texas Hold’Em* cada jogador recebe duas cartas aleatórias do baralho, que é o que chamado de “mão”, sendo essas cartas que apenas o jogador tem acesso, enquanto isso, são

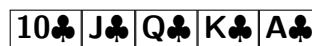
distribuídas ao decorrer do jogo mais cinco cartas de forma comunitárias, as cartas comunitárias são cartas que todos os jogadores tem acesso e podem formar o jogos do pôquer.

2.1.3 As combinações e o desempate

O principal objetivo do pôquer é formar combinações de cinco cartas, sendo um conjunto formado pelas duas cartas da mão do jogador com as cinco cartas comunitárias, sempre descartando as duas cartas de menor valor.

As combinações do pôquer são as seguintes em ordem decrescente de força, sendo a primeira a mais forte e a última a mais fraca:

- *Straight Flush*: É a maior combinação do pôquer, que consiste em cinco cartas em sequência, sendo todas elas do mesmo naipe, sendo em caso de empate, se sobressai o jogador que tiver a maior sequência, que pode ser dita como a que termina com a carta de maior valor, dentro dela se encontra o maior jogo possível do pôquer, que é o *Royal Straight Flush* que consiste na sequência das maiores cartas, do dez até o ás, do mesmo naipe, que é a mostrada no exemplo abaixo:



- *Quadra*: A quadra são quatro cartas de mesmo valor, o desempate é feito pela maior quadra, a quadra pode ser entendida da seguinte forma:



- *Full House*: Consiste no jogador formar uma combinação com uma trinca e um par, o desempate é feito pela maior trinca, caso as trincas sejam iguais, é dada pelo maior par, podendo ser visto da seguinte forma:



- *Flush*: Ocorre quando as cinco cartas da combinação do jogador tem o mesmo naipe, o desempate é feito pela maior carta de cada jogador, caso seja a mesma, a seguinte

maior carta do jogador é o desempate, assim sucessivamente até a quinta carta caso seja necessário, um *flush* pode ser visto como:



- *Straight*: É a sequência de cinco cartas sem importar o naipe das cartas que a compõem, o desempate é feito pela maior sequência, segue um exemplo de *straight*:



- Trinca: É uma combinação formada por três cartas de mesmo valor, o desempate é feito pela maior trinca, a trinca pode ser vista como:



- Dois pares: Consiste no jogador formar dois pares de cartas do mesmo valor, sem importar o naipe, sendo o desempate feito pelo maior par dentre os dois pares, caso o primeiro maior par seja igual entre os jogadores, o segundo par é o quesito de desempate, um exemplo de dois pares é:



- Par: É quando o jogador forma um par de duas cartas de mesmo valor, sem importar o naipe, o desempate é feito pelo maior par, segue um exemplo de par:



- Carta alta: Caso nenhum dos jogadores tenha formado nenhuma combinação, o vencedor é aquele que tiver a maior carta em sua mão, o desempate é feito comparando a maior carta dos jogadores, se for a mesma, a próxima verificada é a segunda maior, assim por diante até a quinta carta de cada um, segue um exemplo de jogo de carta alta:



É válido elucidar que as sequencias no pôquer tem um limite, sempre começando no ás, quando se segue do ás até o cinco, sendo a menor sequencia possível, e sempre terminam no ás, sendo a maior sequencia possível a que vai do dez até o ás, sendo assim as sequencias não são um ciclo, são uma linha fechada.

2.1.4 O desempate

Já foi citado o desempate específico de cada combinação acima, entretanto, em raros casos, quando dois jogadores possuem a mesma combinação, é necessário aplicar outro método de desempate, que é conhecido como *kicker*, considerando o seguinte cenário de empate entre dois jogadores:

Jogador 1 possui: 

Jogador 2 possui: 

Nesse caso os dois jogadores possuem dois pares de mesmos valores, sendo assim, o *kicker* é a maior carta fora da combinação, nesse caso sendo o ás para o jogador 1 e o quatro para o jogador 2, sendo assim, o jogador 1 possui o *kicker* maior, dando a ele a vitória.

2.1.5 As fases de aposta

No pôquer, as apostas seguem o sentido horário dos jogadores, sempre se mantendo nesse sentido. O *Texas hold'Em* possui cinco fases de aposta, que são os momentos que os jogadores podem apostar suas fichas, sendo elas:

- *Pre-Flop*

É quando os jogadores recebem suas duas cartas privadas, sem saber quais as cartas de seus adversários, nessa fase não existem cartas comunitárias, além de ser a única fase de apostas obrigatórias, em que o jogador precisa apostar se desejar participar da rodada.

- *Flop*

São viradas três cartas comunitárias, ocorrendo mais uma rodada de apostas opcionais, que caso um dos jogadores aposte, todos os outros que quiserem se manter na rodada devem apostar também.

- *River*

É virada a quarta carta comunitária, seguindo a mesma lógica de apostas do *flop*.

- *Turn*:

É virada a última carta comunitária, seguindo a mesma lógica de apostas das duas fases

anteriores, após o *turn* é executado o *showdown*, que consiste nos jogadores remanescentes da rodada mostrarem sua mão para definir o vencedor.

2.1.6 As ações do jogador

Dentro de cada fase de apostas, o jogador pode tomar cinco ações, sempre respeitando seu momento de tomar ações, sendo elas:

- *Bet*: É quando o jogador aposta uma quantia de fichas;
- *Call*: Consiste em o jogador pagar uma aposta que outro jogador fez;
- *Check*: Quando o jogador não quer apostar, ou quando não há apostas para serem cobridas, o jogador pode dar *check*, que funciona como uma passada de vez sem penalidades;
- *Raise*: O jogador pode dar um *raise* quando quer aumentar uma aposta que outro jogador fez, sendo que um *raise* deve sempre ser o no mínimo o dobro da aposta que o mesmo quer aumentar;
- *Fold*: Consiste na desistencia do jogador, o mesmo desiste da mão e entrega as cartas, perdendo o valor que apostou anteriormente caso tenha apostado;
- *All-in*: É quando o jogador aposta todas as suas fichas disponiveis, o jogador pode optar pelo *all-in* quando tem uma mão muito forte, como blefe, ou quando não tem a quantidade necessárias de fichas para cobrir a aposta, mas ainda deseja continuar na rodada.

2.1.7 Outros termos do pôquer

Há ainda alguns termos que devem ser explicados, sendo eles o seguinte:

- *Stack*: É a quantidade de fichas que o jogador possui;
- *Pote*: São todas as fichas que já foram apostadas na rodada;
- *Blind*: São apostas obrigatórias rotativas que ocorrem no *pre-flop*, sendo elas:
 - *Small blind*: É metade da aposta minima paga pelo primeiro jogador da rodada na sequencia;
 - *Big blind*: É a aposta total paga pelo jogador exatamente após o jogador que pagou o *Small blind*.

As apostas subsequentes aos *blinds* devem começar pelo valor estabelecido no *big blind*, ou em caso de aumento da aposta, no mínimo o dobro do *big blind*, não podendo ser feita uma aposta menor que o próprio, a menos que o jogador opte por um *All-in*.

2.2 Estatística e o espaço de estados do *Texas Hold’Em*

De acordo com Chen e Ankenman (2006), grande parte das decisões tomadas em uma partida de poquêr, vêm de condições que ainda não foram determinadas, ou seja, de informações que o agente não possui.

É utilizado o exemplo da probabilidade de ser retirado aleatoriamente uma carta específica do baralho, sendo que o mesmo possui 52 cartas, há uma chance $\frac{1}{52}$ de uma carta específica ser retirada de forma aleatória.

Também é possível tomar a quantidade de mãos possíveis no *pre-flop*, tendo em vista que são apenas duas cartas, podemos descrever como:

$$\binom{52}{2} = \underbrace{1326}_{\text{Combinações de cartas na mão do agente}}$$

O que gera 1326 possibilidades de combinações de cartas para cada jogador no *pre-flop*, enquanto as combinações possíveis de cartas comunitárias na mesa, cuja são 5 cartas, pode ser descrito como:

$$\binom{52}{5} = \underbrace{2598960}_{\text{combinações de cartas comunitárias possíveis}}$$

Além disso, existem mais variáveis que devem ser levadas em consideração pelo agente no poquêr, não apenas as próprias cartas, é necessário calcular quais as possíveis cartas dos outros agentes, as possíveis cartas a virem nas cartas comunitárias, a quantidade de fichas, tanto do próprio agente quanto a dos rivais, possibilidade de jogos serem formados, entre outros, tornando assim um espaço extremamente amplo de probabilidades.

2.3 Padrão *MVC*

A etapa de estruturação da API tem como objetivo traduzir os elementos essenciais do jogo de pôquer — incluindo jogadores, mãos, apostas e rodadas — em componentes técnicos adequados ao desenvolvimento de sistemas computacionais. Essa conversão é realizada por meio da modelagem de objetos, definição de *endpoints* e configuração de variáveis que representam o comportamento e as interações do jogo de forma programável e escalável.

A arquitetura da API é concebida segundo o modelo MVC, um padrão arquitetural clássico de engenharia de software que promove a separação clara de responsabilidades, modularidade, coesão e baixo acoplamento entre componentes. Originado na década de 70 por Reenskaug (1979), o MVC foi formalizado como um padrão de *design* no livro “*Design Patterns: Elements of Reusable Object-Oriented Software*” Gamma *et al.* (1994), onde é descrito como uma estrutura tripartite para organizar código em aplicações interativas. Esse modelo divide a lógica da aplicação em três camadas interconectadas, mas independentes:

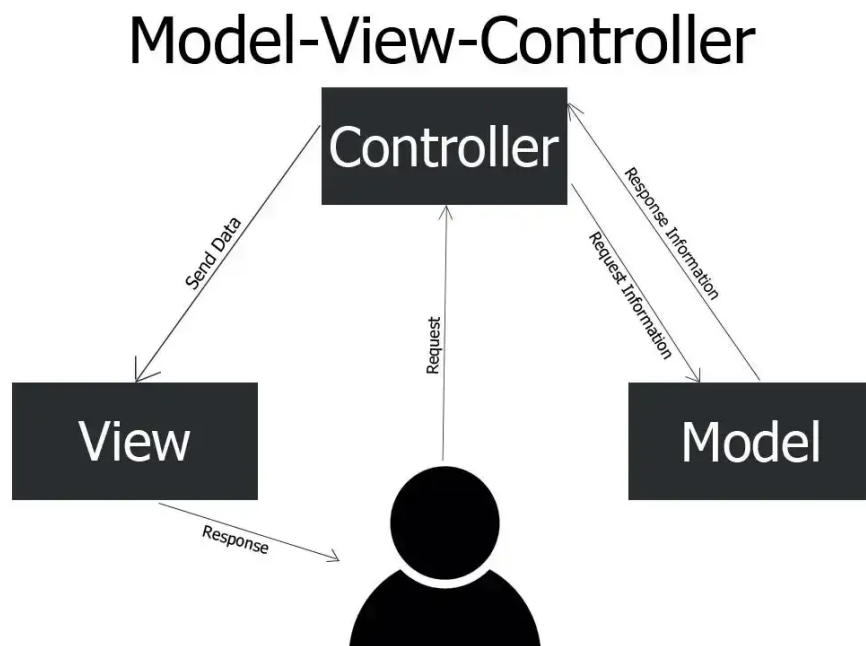


Figura 1 – Arquitetura MVC

- **Model (Modelo):** Representa os dados e a lógica de negócios subjacente, independentemente de como eles são apresentados ou manipulados. No contexto desta API de pôquer, o *Model* encapsula entidades como Jogador, Mão de Cartas, Apostas e Rodada, gerenciando regras do jogo (ex.: validação de apostas válidas, cálculo de probabilidades via Monte Carlo) e persistência de dados (ex.: banco de dados para histórico de mãos). O *Model* notifica as outras camadas sobre mudanças de estado, garantindo que a integridade dos dados seja mantida sem interferência de interfaces ou controles. De acordo com Gamma *et al.* (1994), o *Model* é “o coração da aplicação”, focado em abstrações do domínio problemático, o que facilita testes unitários e reutilização em diferentes contextos.
- **View (Visão):** Responsável pela apresentação e formatação dos dados para o usuário ou sistema externo, sem conter lógica de negócios. Em uma API *RESTful* como esta, a *View* pode ser implementada como respostas *JavaScript Object Notation* (JSON) formatadas

(ex.: *endpoint* /jogo/{id}/estado retornando o status atual da rodada em formato legível), templates *HyperText Markup Language* (HTML) para dashboards de simulação ou integrações com *front-ends web*. A *View* observa o *Model* e se atualiza automaticamente via mecanismos de observador, conforme descrito no mesmo livro do Gamma *et al.* (1994). Isso assegura que alterações no *Model* (ex.: nova aposta realizada) sejam refletidas imediatamente na saída, promovendo interfaces flexíveis e múltiplas *views* para o mesmo conjunto de dados.

- **Controller (Controlador):** Atua como intermediário, processando entradas do usuário (ex.: requisições *Hypertext Transfer Protocol* (HTTP) POST para ações como “apostar” ou “fold”) e orquestrando interações entre *Model* e *View*. No pôquer AI, o *Controller* gerencia fluxos como inicialização de rodadas, chamadas à rede neural para decisões e execução de simulações *Monte Carlo*, direcionando atualizações no *Model* e renderizando a *View* apropriada. Reenskaug (1979) enfatizava que o *Controller* “interpreta comandos do usuário e atualiza o *Model*”, evitando que a *View* ou *Model* lidem diretamente com inputs, o que reduz complexidade e facilita manutenção.

Essa tríade do MVC permite ciclos unidirecionais eficientes: o usuário interage via *Controller*, que modifica o *Model*, que notifica a *View* para atualização. Vantagens incluem escalabilidade (fácil adição de novas *views*), testabilidade (camadas isoladas) e manutenção (mudanças em uma camada não afetam as outras), como validado em *frameworks* modernos como *Spring MVC* (Java) por Hansson (2004). Priorizando esses princípios, a arquitetura da API define módulos específicos para o gerenciamento das etapas do jogo, execução de simulações *Monte Carlo* e integração com a rede neural para tomada de decisão e identificação de comportamentos automatizados. Essa estrutura modular isola funcionalmente cada componente — *Model* para lógica de pôquer, *Controller* para *endpoints* API e *View* para saídas, facilitando manutenção, depuração e expansões futuras.

Além disso, a arquitetura proposta contempla mecanismos de interface entre as camadas MVC, assegurando comunicação eficiente e intercâmbio consistente de dados entre simulação estatística e aprendizado de máquina. Foram adotadas práticas de normalização de dados, controle de versionamento de *endpoints* (ex.: /v1/jogo/{id}) e injeção de dependências para acoplamento frouxo, garantindo estabilidade e compatibilidade.

Essa etapa fundamenta a integridade técnica de todo o projeto, estabelecendo uma base sólida para fases subsequentes de testes, documentação e integração, ao mesmo tempo em que

proporciona clareza estrutural e favorece a reprodutibilidade da pesquisa no contexto acadêmico.

2.4 Inteligência artificial e aprendizado por reforço

O termo Inteligência Artificial (IA) pode ser considerado um grande universo de técnicas, sendo o termo IA o mais amplo deles, englobando vários outros aspectos, a ferramenta de IA que será abordado é o aprendizado por reforço profundo, uma técnica de *Machine Learning*, que consiste em um sistema computacional aprender a executar uma ação, ou conjunto de ações, como é citado em Russell (2010) pode ser entendido como uma criança aprendendo a brincar, ela faz movimentos aleatórios até entender quais movimentos se enquadram melhor para a brincadeira. Para isso tomamos o modelo de aprendizado, já que o mesmo se enquadra melhor para a criação de uma agente capaz de jogar o *Texas Hold’Em*, dito isto, o agente pode ser baseado em três tipos de aprendizado de máquina, como destacado em Russell (2010), sendo alguns deles os seguintes:

- **Supervisionado:** O agente observa padrões e/ou entradas e saídas que são disponibilizadas para que o mesmo aprenda da melhor maneira como usar aqueles dados para executar suas ações;
- **Não-supervisionado:** O agente trabalha sem necessariamente receber dados para complementar o modelo, como por exemplo um agente de tráfego de táxi, usando um modelo de aprendizado não supervisionado, ele pode usar diretrizes que o mesmo “percebeu” para ditar “Dias com tráfego bom” ou “Dias com tráfego ruim”;
- **Aprendizado por reforço:** O agente aprende por uma série de recompensas ou penalidades, como o agente utilizado neste trabalho é baseado em um aprendizado de máquina por reforço, a maneira do próprio receber as recompensas e punições é feita por conta das fichas do poquêr, sendo ganhar fichas uma recompensa e perder uma penalidade.

Tendo os diferentes tipos de aprendizado mencionados, e como dito acima, o selecionado para este trabalho é um aprendizado de máquina por reforço, dado tal explicação cabe explicar os componentes que compõem um aprendizado por reforço, sendo eles:

- **O agente:** É a inteligência artificial propriamente dita, o agente toma as decisões baseadas no ambiente em que o mesmo está enquadrado;
- **O ambiente:** Se trata de onde o agente vai trabalhar, no caso deste trabalho, se referencia à mesa de poquêr *Texas Hold’Em*, seguindo as regras estabelecidas do jogo;

- **O estado:** Pode ser dito como os elementos que estão à disposição do agente, e que neste caso, são variáveis dependendo do momento do ambiente, neste caso as fichas e as cartas tomam o papel de estado;
- **A ação:** É definida como as ações que o agente pode tomar para obter o melhor resultado possível, neste caso temos as seguintes ações: *Fold*, *Call* e *Raise*;
- **A recompensa:** Como o aprendizado utilizado é o aprendizado por reforço, o sistema de recompensa é essencial para moldar o agente da melhor maneira possível, no caso deste trabalho o sistema de recompensa se baseia no lucro ou prejuízo de fichas, sendo ganhar fichas uma recompensa, enquanto perder fichas, ou deixar de ganhar, gera uma penalidade.

2.5 Método de *Monte Carlo* aplicado a uma rede neural jogadora de pôquer

O método de *Monte Carlo* é uma classe de técnicas numéricas baseadas em amostragem aleatória para estimar quantidades de interesse em problemas cuja solução analítica é inviável ou extremamente custosa. Historicamente, o método foi formalizado na década de 1940 no contexto do Projeto Manhattan em Los Alamos, quando Stanislaw Ulam e John von Neumann passaram a utilizar amostragens aleatórias para resolver problemas complexos de transporte de nêutrons e física nuclear, em substituição a cálculos combinatórios e integrais de alta dimensionalidade Leonelli (2021). O nome “*Monte Carlo*” foi sugerido por Nicholas Metropolis em referência ao famoso cassino de Mônaco, ressaltando a analogia com experimentos de sorteio e jogos de azar.

De forma geral, o objetivo do método de *Monte Carlo* é aproximar uma quantidade esperada, como uma integral ou esperança matemática, por meio da média de amostras geradas aleatoriamente. Dado um vetor aleatório X com distribuição $p(x)$ e uma função de interesse $f(x)$, deseja-se estimar a esperança

$$\mu = \mathbb{E}[f(X)] = \int_{-\infty}^{\infty} f(x) p(x) dx$$

Em muitos casos, essa integral não admite forma fechada ou é impraticável de calcular diretamente, especialmente em espaços de alta dimensão. O método de *Monte Carlo* substitui

então essa integral pela média amostral

$$\hat{\mu}_N = \frac{1}{N} \sum_{i=1}^N f(X_i)$$

em que $X_1, X_2, \dots, X_N$ são amostras independentes e identicamente distribuídas segundo $p(x)$. Pelo Teorema da Lei dos Grandes Números, $\hat{\mu}_N$ converge quase certamente para μ quando $N \rightarrow \infty$, e o Erro Padrão da estimativa diminui aproximadamente com a ordem de $1/\sqrt{N}$ (Rubinstein; Kroese, 2008). Essa propriedade torna o método de Monte Carlo particularmente útil para problemas de alta complexidade, onde métodos determinísticos teriam custo proibitivo.

Do ponto de vista prático, um algoritmo básico de Monte Carlo para estimar μ pode ser descrito em três passos (Rubinstein; Kroese, 2008):

1. Gerar N amostras pseudoaleatórias $X_1, X_2, \dots, X_N$ a partir da distribuição alvo $p(x)$ ou de uma distribuição equivalente (por exemplo, via transformações, amostragem por rejeição ou métodos de cadeia de Markov).
2. Avaliar a função de interesse $f(X_i)$ para cada amostra gerada.
3. Calcular a média amostral $\hat{\mu}_N$ como estimativa da quantidade de interesse, juntamente com estimativas de variância e intervalos de confiança.

Rubinstein e Kroese destacam que, embora o método seja conceitualmente simples, boa parte da pesquisa em *Monte Carlo* concentra-se em aprimorar a geração de amostras e reduzir a variância das estimativas (por exemplo, via amostragem estratificada, variáveis antitéticas ou amostragem por importância) Rubinstein e Kroese (2008).

2.5.1 Aplicação do método de *Monte Carlo* em jogos e pôquer

Jogos de cartas como o pôquer são domínios clássicos para aplicação de *Monte Carlo*, pois envolvem incerteza combinatória (cartas ocultas, cartas futuras), múltiplos agentes e grandes espaços de estados. Em pôquer *Texas Hold'em*, por exemplo, um jogador conhece apenas suas cartas privadas e as cartas comunitárias já reveladas, mas desconhece as cartas dos oponentes e as cartas que ainda serão abertas. Isso torna difícil calcular exatamente a probabilidade de vitória ou o valor esperado de uma ação (pagar, aumentar, desistir) em tempo real.

O método de *Monte Carlo* é empregado para contornar essa dificuldade por meio de simulações do futuro da mão. A ideia básica é:

1. Fixar o estado atual da mão: cartas próprias do agente, cartas comunitárias já reveladas, ações anteriores dos jogadores e tamanho dos potes.

2. Amostrar aleatoriamente, muitas vezes, as cartas ocultas dos oponentes e as cartas futuras que ainda serão reveladas, respeitando o baralho remanescente (ou seja, sem reposição e sem conflitos com cartas já visíveis).
3. Para cada cenário simulado, completar a mão até o final (showdown ou desistências) e avaliar o resultado: vitória, derrota, empate, lucro ou prejuízo monetário.
4. Estimar, por meio da média dos resultados simulados, a probabilidade de vitória ou o valor monetário esperado (EV) de uma ação específica.

Diversos trabalhos em pôquer computacional adotam variações desse esquema, tanto para avaliar a força de uma mão quanto para apoiar decisões estratégicas Kleij (2010), Broeck, Driessens e Ramon (2009), Giacomo e Martin (2015). Por exemplo, Van der Kleij construiu um jogador de pôquer computacional que utiliza busca em árvore por *Monte Carlo*, MCTS, combinada com modelagem de oponentes para estimar valores esperados de ações em diferentes estágios do jogo Kleij (2010). Em outro estudo, Van den Broeck e colaboradores aplicam MCTS em pôquer no-limit, utilizando simulações para explorar o espaço de decisões possíveis e discretizar tamanhos de aposta, reduzindo o custo computacional sem perder competitividade Broeck, Driessens e Ramon (2009). Esses trabalhos demonstram que, mesmo em jogos de informação imperfeita, o uso sistemático de simulações aleatórias permite aproximar o valor das ações de forma eficaz.

Em contextos mais heurísticos, *Monte Carlo* também é amplamente utilizado para validar estratégias ou heurísticas de decisão em pôquer. Por exemplo, estudos utilizam simulações de centenas de milhares ou milhões de mãos para estimar a probabilidade de ocorrência de determinados tipos de mãos ou para verificar o desempenho de regras fixas de jogo ao longo do tempo Giacomo e Martin (2015). Essa mesma lógica de simulação extensiva é diretamente aproveitada em arquiteturas de inteligência artificial, nas quais a estimativa robusta de probabilidades e valores esperados é um insumo fundamental para o treinamento de modelos.

2.5.2 Integração do método de *Monte Carlo* com redes neurais para pôquer

No contexto desta pesquisa, o método de *Monte Carlo* é empregado como um mecanismo de estimação estatística que gera alvos ou sinais de treinamento para uma rede neural projetada para jogar pôquer. A integração entre *Monte Carlo* e aprendizado de máquina segue a linha de trabalhos em aprendizado por reforço (Reinforcement Learning), nos quais estimativas de retorno obtidas a partir de episódios completos são utilizadas para atualizar funções de valor

e políticas.

Em termos conceituais, a rede neural recebe como entrada uma representação de estado do jogo (por exemplo, codificação das cartas próprias, cartas comunitárias, posição na mesa, tamanho dos potes, histórico recente de apostas e características dos oponentes) e produz como saída:

- estimativas de valor, representando o valor esperado da posição atual (por exemplo, em unidades monetárias ou em probabilidade de vitória); e/ou
- distribuições de probabilidade sobre ações (por exemplo, dobrar, pagar, aumentar), representando uma política estocástica de jogo.

Para treinar essa rede, o método de Monte Carlo é utilizado em duas frentes principais:

2.5.2.1 Geração de rótulos a partir de simulações

Uma abordagem direta consiste em usar simulações de *Monte Carlo* para rotular estados com estimativas de valor ou probabilidade de vitória. O procedimento típico é:

1. Selecionar um conjunto de estados de pôquer (reais ou gerados artificialmente).
2. Para cada estado, executar um grande número de simulações Monte Carlo, amostrando cartas ocultas dos oponentes e cartas futuras, e jogando a mão até o fim sob uma política fixa ou sob um modelo de oponentes.
3. Calcular, para cada estado, a média dos resultados simulados (por exemplo, lucro líquido médio ou probabilidade estimada de vitória).
4. Utilizar esses valores médios como alvos supervisionados para treinar a rede neural, minimizando uma função de perda adequada (como erro quadrático médio para valores ou entropia cruzada para probabilidades).

Essa estratégia transforma o problema em um cenário de aprendizado supervisionado onde *Monte Carlo* fornece um especialista para a rede, à semelhança do que Sutton e Barto (2018) discutem para métodos de *Monte Carlo* em aprendizado por reforço, nos quais o retorno total observado em um episódio serve como alvo para a função de valor.

2.5.2.2 Métodos de *Monte Carlo* em aprendizado por reforço

Outra forma de integração é empregar diretamente métodos de *Monte Carlo* em aprendizagem por reforço, em que a própria interação da rede com um ambiente simulador de pôquer gera episódios de jogo. Nesse cenário:

- Cada episódio corresponde a uma sequência de estados, ações e recompensas (por exemplo, ganhos e perdas em fichas).
- Ao final do episódio, calcula-se o retorno total G_t a partir de cada estado s_t :

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k,$$

em que R_k é a recompensa na etapa k , $\gamma \in [0, 1]$ é o fator de desconto e T é o tempo final do episódio.

- Esses retornos G_t são utilizados para atualizar os parâmetros da rede, seja como alvos para uma função de valor $V(s)$, seja como sinal de reforço para ajustar uma política parametrizada $\pi_\theta(a|s)$.

Sutton e Barto (2018) mostram que tais métodos de *Monte Carlo* permitem aprender diretamente a partir da experiência, sem a necessidade de um modelo completo das dinâmicas do ambiente, o que é particularmente atraente em domínios complexos como o pôquer, em que a modelagem exata dos oponentes e das distribuições de cartas é difícil. Na prática, pode-se combinar simulações de *Monte Carlo* com técnicas como busca em árvore MCTS ou amostragem de políticas para explorar as consequências de diferentes ações antes de atualizá-las na rede neural.

2.5.3 Aspectos práticos e considerações metodológicas

A aplicação do método de *Monte Carlo* em pôquer associado a redes neurais envolve algumas decisões metodológicas importantes:

- **Número de simulações (N):** Há um compromisso entre custo computacional e precisão estatística. Valores de N muito baixos resultam em estimativas ruidosas, enquanto valores muito altos aumentam o tempo de treinamento. Em geral, escolhe-se N de forma a garantir erros padrão aceitáveis para as estimativas de valor, muitas vezes com base em experimentos preliminares como dito por Rubinstein e Kroese (2008).
- **Modelagem de oponentes:** Em jogos de informação imperfeita, as simulações devem refletir crenças realistas sobre o comportamento dos oponentes. Trabalhos em pôquer computacional utilizam modelos paramétricos de oponentes, aprendidos a partir de dados, para amostrar ações e cartas de forma consistente com estilos de jogo observados por Kleij (2010), Broeck, Driessens e Ramon (2009).
- **Representação de estados:** A qualidade das estimativas de *Monte Carlo* depende tam-

bém da expressividade da representação utilizada na rede neural. Entradas que resumem adequadamente a informação relevante (por exemplo, tamanhos relativos de pilha, posição, textura do bordo e histórico de ações) favorecem o aprendizado de padrões estáveis a partir dos retornos simulados.

- **Redução de variância:** Técnicas clássicas de redução de variância podem ser adaptadas ao contexto de pôquer, por exemplo, estratificando simulações por tipo de mão inicial ou por determinadas configurações de bordo, de modo a obter estimativas mais precisas sem necessariamente aumentar N como notado por Rubinstein e Kroese (2008).

Dessa forma, o método de *Monte Carlo* funciona como um elemento estruturante, fornecendo a base estatística para a estimativa de probabilidade de vitória, valor esperado de ações e retornos de aprendizado por reforço.

2.6 Redes neurais artificiais e *Q-Learning*

Uma Rede Neural Artificial (RNA) emula o funcionamento do cérebro humano, neste aspecto, há varias formas de moldar uma RNA, a que será citada será uma rede neural artificial perceptron multicamadas, como dito em Bengio *et al.* (2017), a mesma pode ser definida como uma função que mapeia um conjunto de entradas e retorna uma saída, utilizando para isso, neurônios, que servem para definir as saídas baseadas nas entradas, pesos, que são parâmetros passados para os neurônios para tomarem a melhor decisão e funções de ativação que, como o nome se refere, ativa os neurônios.

O *Q-Learning* é um algoritmo de aprendizado por reforço que associa valores de qualidade, *Q-Values*, às ações disponiveis em determinado estado, de modo a estimar a melhor decisão. O *Q-Learning* trabalha usando a *Q-Table*, que consiste em uma tabela que suas linhas são todos os estados possíveis do jogo, enquanto as colunas são todas as ações possíveis, a rede verifica o estado e confere qual ação tem o maior *Q-Value* e executa a devida ação.

Para calcular o valor ideal do *Q-Value*, como citado por Mnih *et al.* (2015), é utilizada a fórmula de Bellman, cuja é a seguinte:

$$Q_{i+1}(s, a) = r + \gamma \max Q_i(s', a')$$

Tal fórmula calcula o *Q-Value*, cada termo significa o seguinte:

- $Q_{i+1}(s, a)$: Significa o valor a ser calculado, onde o s representa o estado que o agente se encontra, enquanto o a representa a ação a ser tomada, o $i+1$ representa o novo valor a

ser adicionado;

- r : Representa a recompensa imediata por tomar tal ação;
- γ : Representa o fator de desconto, variando entre 0 e 1, sendo 0 quando o agente leva em consideração apenas a recompensa imediata(r);
- $\max Q_i(s', a')$: Esta função representa o maior Q -Value possível, tomando o estado futuro (s') tomando a melhor ação possível (a').

Entretanto um algoritmo de Q -Learning tradicional funciona perfeitamente com jogos mais simples e com menos ações e estados a serem verificados, jogos como jogo da velha ou labirintos são exemplos simples para se utilizar o Q -Learning tradicional, porém o *Texas Hold'Em*, por conta da quantidade de combinações de cartas, tamanho do pote, quantidade de fichas do jogador e ações a serem tomadas, criaria uma tabela que beiraria a infinidade de estados que poderiam ser tomados, tornando ineficaz e praticamente impossível a criação de uma Q -Table para um jogo de pôquer, tendo em vista que nenhum computador moderno teria capacidade de memória para alocar uma tabela tão extensa.

2.7 Deep Q-Network

O *Deep Q-Network* é uma variação do algoritmo de Q -Learning padrão, o mesmo é usado quando a quantidade de estados beira o infinito, como falado por Mnih *et al.* (2015), já que o mesmo usa de uma rede neural para fazer a predição do Q -Value.

Como citado anteriormente, esta abordagem não é eficiente já que é calculada sem nenhuma generalização, sendo assim, o *Deep Q-Network* foi criado para que fosse possível que a rede neural fizesse a predição sem necessidade de criar uma tabela grande demais até mesmo para ser alocada na memória, assim podendo usar uma equação para se calcular o valor aproximado do Q -Value, usando a equação:

$$L_i(\theta_i) = \mathbb{E}_{\underbrace{(s, a, r, s') \sim U(D)}_{\text{Exp. Replay}}} \left[\left(r + \underbrace{\gamma \max_{a'} Q(s', a'; \theta_i^-)}_{\text{Alvo}} - \underbrace{Q(s, a; \theta_i)}_{\text{Predição}} \right)^2 \right]$$

Esta é uma fórmula de perda, descrita por Mnih *et al.* (2015), que pode ser desmembrada e entendida como:

- $L_i(\theta_i)$: é a taxa de perda na iteração i , considerando os pesos atuais θ_i da rede neural;

- $\mathbb{E}_{(s,a,r,s') \sim U(D)}$: Se refere ao *Experience Replay*, que são jogos anteriores guardados na memória;
- $r + \gamma \max_{a'} Q(s', a'; \theta_i^-)$: É a equação de Bellman, porém utilizando iterações passadas otimizadas pela fórmula de perda, representadas por θ_i^- , sendo utilizada como alvo;
- $Q(s, a; \theta_i)$: Representa a predição feita de forma cega pelo agente, utilizando os pesos θ_i ;
- Sendo a diferença entre o alvo e a predição elevados ao quadrado, para evitar números negativos, e dar maior peso a resultados ruins.

O DQN foi desenvolvido por Mnih *et al.* (2015) para ensinar uma Inteligência artificial para jogar Atari, tornando-se um marco histórico para o desenvolvimento de Redes Neurais utilizando o *Q-Learning*.

2.8 Exploração vs. Exploração (*Epsilon-Greedy*)

O agente necessita aferir a melhor jogada possível do *Texas Hold'Em*, para isso ele deve explorar suas opções de jogada, o mesmo inicialmente não tem conhecimento para determinar a melhor jogada possível, logo o agente deve escolher aleatoriamente sua jogada, antes de conseguir explorar as melhores jogadas possíveis.

Para isso, foi usada a tática ϵ -*Greedy* (Epsilon-Guloso), como descrita em Sutton e Barto (2018), o ϵ -*Greedy* consiste em uma tática em que o coeficiente de exploração ϵ determina a aleatoriedade das ações, no caso deste trabalho, o mesmo é iniciado em 1.0, determinando exploração total, ou seja, uma ação completamente aleatória, que vai decaindo ao decorrer das jogadas, chegando próximo de 0.0, assim maximizando a exploração. Sendo assim, reduzindo a complexidade, já que não é necessário encontrar a melhor solução possível, necessitando apenas encontrar uma que atenda o interesse imediato.

2.9 Replay de Experiência (*Experience Replay*)

Pela extensa quantidade de estados que o *Texas Hold'Em* pode tomar, alguns estados se tornam semelhantes entre si, o que pode ocasionar um ótimo local para a Rede Neural, isso é denotado por Mnih *et al.* (2015), quando é citado que casos semelhantes “viciam” nas ações, tornando o agente enviesado.

Para contornar este problema foi utilizado o *Experience Replay* (Replay de experiência), também citado em Mnih *et al.* (2015), o *Experience Replay* consiste em criar lotes de memórias

aleatórias do passado para serem usados antes do agente decidir a jogada, servindo como parte do treino, e assim evitando enviesamentos do agente, deixando o mesmo mais robusto e mais assertivo com a proposta do jogo.

3 TRABALHOS RELACIONADOS

3.1 Abordagens baseadas em busca

Antes de redes neurais mais profundas serem amplamente utilizadas para produção de projetos envolvendo o *Texas Hold’Em*, eram utilizadas abordagens mais voltadas para busca em árvore, como feito por Kleij (2010) e por Broeck, Driessens e Ramon (2009), que utilizaram o algoritmo de *Monte Carlo* para explorar os estados do jogo, porém como ambos focavam majoritariamente na modelagem e busca em árvore de maneira mais bruta, enquanto este trabalho é mais focado na tomada de decisões em si, porém o *Monte Carlo* é usado de forma semelhante nas fase de *Pré-Flop* e de *Flop* para calcular o valor de equidade.

3.2 Avanço em *Deep Reinforcement Learning*

Com o passar do tempo, técnicas de Aprendizado por Reforço Profundo se tornaram mais utilizadas para desenvolvimento de agentes inteligentes capazes de jogar o *Texas Hold’Em*, como feito por Moravčík *et al.* (2017), que desenvolveu um agente capaz de vencer jogadores profissionais de poquêr *Heads-Up*, outra categoria do jogo quem consiste em um contra um direto, utilizando clusters poderosos carregados de dados, enquanto este trabalho é mais focado na intuição dos jogos.

Enquanto a maioria dos trabalhos realizados para criação de agentes para jogar poquêr jogam apenas a categoria *Heads-Up*, e enfrentam inúmeros problemas de ótimo local, assim como Mnih *et al.* (2015) tentou desenvolver um agente utilizando apenas uma DQN puro para jogar o *Texas Hold’Em*, mas atingiu ótimos locais de desistências infinitas assim como este trabalho, tendo isso em vista Heinrich e Silver (2016) desenvolveu um agente capaz de jogar *Texas Hold’Em* utilizando táticas para contornar as fraquezas do DQN, enquanto a tática utilizada neste trabalho foi a de penalidade por jogar passivamente.

Também pode ser citado o artigo de Brown e Sandholm (2019) que desenvolveu um agente que se sobressaiu em uma mesa com 6 jogadores, demonstrando um domínio completo sobre ambientes multi-jogadores, o agente jogava de maneira que não permitia que os outros jogadores explorassem suas falhas, tendo um comportamento similar ao agente desenvolvido neste trabalho, que explora a falha dos outros jogadores para tomar as melhores decisões possíveis.

3.3 Diferença arquitetural

Enquanto os trabalhos citados anteriormente dispunham de um extenso maquinário para o desenvolvimento dos agentes, este trabalho se voltou mais para o fator de exploração do agente, fazendo com que o próprio aprendesse com suas próprias ações, além de utilizar modulação de recompensa para deixar o agente mais preciso, o penalizando por fazer uma jogada desnecessariamente passiva, ou o recompensando por fazer uma jogada embasadamente agressiva, fazendo com que o jogador tenda a seguir mais fielmente a dinâmica do *Texas Hold'Em*.

Também pode ser citada a questão de desempate, que costuma ser resolvida por buscas em árvores complexas e mais custosas, este projeto desenvolveu um motor de base 15 capaz de efetuar os desempates de forma concisa e coerente, deixando o programa relativamente mais rápido. A linguagem utilizada para o desenvolvimento também é diferente da maioria utilizada, já que foi utilizado o *JavaScript*, utilizando do *TensorFlow.js* e *Node.js* como uma alternativa no lugar da comumente usada *Python*.

4 METODOLOGIA

4.1 Tecnologias e Ferramentas Utilizadas

A linguagem utilizada para este trabalho foi a *JavaScript*, usando os recursos do *Node.js*, já que foram executados longos *loops* para simular milhares de mãos de *Texas Hold'Em*, tendo em vista que a quantidade de iterações poderia gerar travamentos na execução, as funções assíncronas do *Node.js* (*await/async*) tornavam a execução do programa mais eficiente.

Para desenvolvimento da Rede Neural foi utilizado o *TensorFlow.js*, dispensando a necessidade de fazer uma conexão entre *JavaScript* e *Python* caso fosse usado apenas o *TensorFlow* tradicional, sendo utilizado como referenciado por Smilkov *et al.* (2019), tendo em vista que o *TensorFlow.js* tem funções nativas de DQN, a mesma foi a mais adequada para o trabalho.

A plotação gráfica foi feita utilizando do *Chart-Js-Node-Canvas*, API que permite a criação dos gráficos, sendo utilizada para plotar o desempenho do agente, levando como parâmetro o ganho/perda de fichas, também representando a queda do ϵ .

4.2 Arquitetura do ambiente e motor de regras

4.2.1 Estrutura MVC

Este trabalho é todo arquitetado utilizando o padrão de arquitetura MVC, como é possível verificar no repositório de Grangeiro (2026), a pasta *Models* comporta as classes referentes ao *Texas Hold'Em*, sendo elas a classe “carta” que representa as cartas do baralho utilizadas no poquê, contendo o Naípe e o Valor da carta, e a classe “jogador” representa o agente, que contém as cartas disponíveis para o jogador, seu *Stack* e a ação que o jogador vai tomar. A pasta *controllers* possui o controlador responsável por gerenciar as fases de apostas, sendo elas as descritas no código como:

```
const fases = [
  { nome: "Pre-Flop", cartas: 0 },
  { nome: "Flop", cartas: 3 },
  { nome: "Turn", cartas: 1 },
  { nome: "River", cartas: 1 }
];
```

Código-fonte 1 – Fases gerenciadas pelo jogoController.js

A pasta *Services* vai conter toda a parte estatística e da inteligência do agente, podendo ser considerada o cérebro das atividades do projeto, contendo todas as funções referentes ao baralho, a Inteligência Artificial, os gráficos, as combinações e as estatísticas geradas pelo *Monte Carlo*. Por fim, a pasta *Views* gerencia aquilo referente ao que o usuário vê durante o processamento do programa, no caso deste trabalho, controla a barra de carregamento que pode ser vista no terminal de comandos, além como as funções responsáveis por escrever informações em um arquivo de texto e uma função responsável por escrever as cartas de maneira mais compreensível para o usuário.

4.2.2 Motor de avaliação - Base 15

Um dos principais desafios para se modular as regras do poquê são a valorização das jogadas, porém é possível superar esse desafio usando lógica simples, dando valores de força para as jogadas, entretanto cai-se em outra barreira, os empates são ocasiões incomuns, mas que são um grande desafio de serem resolvidos usando uma simples lógica de valores para as jogadas. Mesmo que o poquê disponha do *Kicker*, sendo necessário verificar qual categoria se sobressai, para seres humanos é fácil distinguir quais pares são maiores que outros, por exemplo, mas para o computadores, se o par tem sempre peso 2, independente do valor dos pares, eles serão iguais.

Para contornar isso, foi elaborado um sistema de *score*(pontuação) de base 15, tomando o Ás como valor 14, a base 15 supre todas as necessidades do sistema, garantindo que não haverá possibilidades maiores que o ás, para calcular esse *score* é usada a seguinte fórmula:

$$\text{Score} = (\text{Categoria} \cdot 15^5) + \sum_{i=1}^5 (\text{Carta}_i \cdot 15^{5-i})$$

Esta fórmula calcula o score multiplicando o valor da categoria da jogado por 15^5 , sucedido pela soma das 5(cinco) cartas que o jogador tem à sua disposição, cada uma multiplicada por uma exponenciação de base 15(quinze). A função do código que faz esse cálculo se encontra no *combinacaoService.js*, cuja é:

```

const calcularScore = (categoria, v1 = 0, v2 = 0, v3 = 0, v4 = 0,
  v5 = 0) => {
  return categoria * 759375 + v1 * 50625 + v2 * 3375 + v3 * 225
    + v4 * 15 + v5;
};

```

Código-fonte 2 – Função de calculo do Score

Para melhor compreensão do motor base 15 e de como ele é utilizado de forma real no código, segue o exemplo de calculo para a categoria *Dois Pares*:

```

if (pares.length >= 2) {
  let k = kickers([pares[0], pares[1]]);
  return calcularScore(3, pares[0], pares[0], pares[1], pares
    [1], k[0]);
}

```

Código-fonte 3 – Função de score de Dois Pares

Nesse caso, o primeiro parâmetro da função *calcularscore* se refere a força da categoria, no caso de Dois pares a força é 3(três), seguido pelo par maior duas vezes como pode ser notado em *pares[0]*, *pares[0]* para ocupar os multiplicadores mais fortes, sendo esses 15^4 e 15^3 , seguido pelos dois pares menores, que consequentemente recebem os multiplicadores mais fracos, e por fim o *Kicker*, que consiste nas cartas de desempate, nesse caso sendo a última carta disponível do agente.

Sendo assim, na rara ocasião que dois jogadores possuam Dois Pares cada um e o mesmo *Kicker*, o programa é capaz de definir o que se sobressai, dando a vitória para o jogador com a maior jogada.

4.3 Modelagem do agente inteligente (DQN)

4.3.1 Arquitetura e espaço de estados

Tendo em vista que o *TensorFlow.js* foi a API selecionada para este trabalho, o mesmo foi arquitetado com duas camadas ocultas de 16 neurônios com ativação *Rectified Linear Unit* (ReLU), que cortam quaisquer números negativos que possam entrar na rede, além de uma

camada de saída com 5 neurônios de ativação linear, já que o *Q-Learning* retorna um score absoluto, ao invés de uma probabilidade, tudo isso pode ser denotado no seguinte trecho do código:

```
createModel() {
  const model = tf.sequential();
  model.add(tf.layers.dense({ units: 16, inputShape: [4],
    activation: 'relu' }));
  model.add(tf.layers.dense({ units: 16, activation: 'relu' }));
  ;
  model.add(tf.layers.dense({ units: 5, activation: 'linear' }));
  model.compile({ optimizer: 'adam', loss: 'meanSquaredError' });
  return model;
}
```

Código-fonte 4 – Camadas do modelo

Além disso, o estado do agente é composto por 4(quatro) parâmetros, sendo eles o *equityMC*, que é o valor de equidade retornado pelo *Monte Carlo*, o *pote*, que consiste na quantidade de fichas já apostadas na rodada, a *apostaParaCobrir* que representa quanto o agente deve pagar para continuar na rodada, por fim, o parâmetro *minhasFichas* que diz quantas fichas disponíveis o jogador tem. Entretanto, o valor da equidade se encontra entre 0 e 1, enquanto os parâmetros baseados em fichas podem variar de 0 até 1000, sendo necessária uma normalização desses parâmetros, para não haverem conflitos no processamento do estado, é possível ver os parâmetros e a normalização no trecho a seguir:

```
obterEstado(equityMC, pote, apostaParaCobrir, minhasFichas) {
  // Normalizacao basica para ajudar a rede
  return [equityMC, pote / 1000, apostaParaCobrir / 1000,
    minhasFichas / 1000];
}
```

Código-fonte 5 – Estado do jogador

4.3.2 Espaço de ações

Considerando que as apostas no *Texas Hold'Em* são o fator mais importante para o jogo, é extremamente importante que o agente consiga executá-las de maneira consistente, entretanto, como no jogo tradicional as apostas tem valores extremamente imprecisos, podendo variar entre 1 até o todas as fichas disponíveis do jogador, e isso se tornaria um empecilho para o *Q-Learning*, sendo assim, foi feita uma discretização das ações.

Como citado anteriormente, a rede neural possui 5 neurônios de saída, que podem ser ditos como as ações que o agente toma, sendo eles os seguintes:

- *Fold*: O agente desiste da mão;
- *Check/call*: O agente paga as apostas que vem na mesa, ou passa a vez caso não tenham apostas à serem pagas;
- *Raise Leve*: O agente adiciona mais 150 fichas na aposta que vem para ele pagar;
- *Raise Forte*: O agente adiciona mais 300 fichas na aposta que vem para ele pagar;
- *All In*: O agente aposta todas as suas fichas disponíveis.

4.3.3 Modelagem das recompensas

As recompensas são a principal forma de se treinar um agente por reforço, seguindo as regras do *Texas Hold'Em*, as recompensas podem ser dadas simplesmente pelo lucro ou prejuízo de fichas, entretanto apenas usar o lucro/prejuízo do agente pode encaminhá-lo para ótimos locais, como dito por Russell (2010), um ótimo local é quando o agente “descobre” a melhor forma para o que ele aprendeu, mesmo que fuja do ótimo global, que seria a tomada de decisão ideal, no caso deste trabalho, o agente atingiu um ótimo local em que o mesmo sempre desistia de todas as mãos, assim ele não perdia fichas. Para contornar isso, foi utilizada a estratégia de *Blinds*, que são apostas rotativas no *Texas Hold'Em* que forçam o agente a apostar na primeira rodada, o que resolveu o ótimo local do fold na primeira fase, porém, o agente aprendeu que caso apenas desse *Check*, também perderia o mínimo de fichas possíveis. Sendo assim, foi necessário a implementação de uma penalização por passividade, associado de uma bonificação por agressividade, como denotado por Ng, Harada e Russell (1999), sendo feita da seguinte maneira:

```

if (comunitarias.length > 0) {
    // E a IA tem cartas muito fortes (ex: +60% de chance)
    if (equity > 0.60) {
        // Mas ela escolhe Fold (0) ou Check/Call (1)
        if (acao === 0 || acao === 1) {
            jogador.penalidadePorPassividade = (jogador.
                penalidadePorPassividade || 0) - 20;
        }
        // Se ela escolhe ser agressiva (2, 3 ou 4) com carta
        // forte, ganha um bonus
        else if (acao >= 2) {
            jogador.penalidadePorPassividade = (jogador.
                penalidadePorPassividade || 0) + 15;
        }
    }
}

```

Código-fonte 6 – Sistema de penalidade por passividade

Como pode ser visto, caso o agente tenha um *equity* superior a 60%, o mesmo é penalizado em 20 pontos caso escolha a ação de *Fold* ou de *Check*, enquanto recebe uma bonificação de mais 15 quando faz uma jogada agressiva, como aumentar a aposta ou até mesmo dar *all-in*. Sendo assim, o agente evita os ótimos locais mencionados, além de deixar o sistema de recompensas mais robusto, fazendo que o agente seja mais assertivo conforme aprende a jogar o jogo

4.4 Cenários de simulação e Treinamento

4.4.1 O ambiente da mesa

O ambiente foi configurado para comportar 4 agentes que fazem o papel de jogador, cada um iniciando com um *Stack efetivo*(fichas iniciais) de 1000 fichas, com a existência do *Small Blind* e do *Big Blind*, sendo apostas obrigatórias rotativas que os jogadores são obrigados a pagar quando são selecionados, cobrindo todos os jogadores da mesa.

4.4.2 A execução dos treinos

O treinamento ocorre por grandes lotes contínuos de partidas, com exemplo de 10.000 mãos sendo feitas, cada mão podendo ser repetida pela RNA, usando assim uma simulação em vários cenários, como descrito por Leonelli (2021), aplicando os *Experience Replay* utilizando lotes de 32 memórias no final de todas as mãos.

4.4.3 O decaimento do *Epsilon*

Sendo usado como fator de aleatoriedade nas primeiras mãos, já que o agente não sabe como jogar no início, o ϵ é iniciado em 1.0, definindo aleatoriedade total no começo das simulações, decaindo em 0.5% a cada mão, chegando até o valor mínimo de 0.01, esse decaimento é definido no seguinte trecho de código:

```
this.epsilon = 1.0;
this.epsilonMin = 0.01;
this.epsilonDecay = 0.995;
```

Código-fonte 7 – Variáveis responsáveis pelo controle do epsilon

Tendo isso em vista, o ϵ é decrementado no fim de cada simulação da seguinte maneira:

```
if (this.epsilon > this.epsilonMin) {
    this.epsilon *= this.epsilonDecay;
}
```

Código-fonte 8 – Decaimento do epsilon

5 RESULTADOS

5.1 Validação do ambiente e do motor de regras

Tomando um ambiente em que foram executadas 10.000 simulações de mãos do *Texas Hold'Em*, cada mão sendo repetida 10 vezes, o motor de base 15 executou o trabalho de maneira consistente, classificando todos os jogos de fazer os desempates de maneira correta e robusta, isso pode ser atestado observando o jogo 20 das simulações, que é o que consta no apêndice-a.

Esta simulação demonstra o funcionamento do motor base 15, já que o *Jogador[0]* e o *Jogador[2]* possuíam dois pares, porém o *Jogador[0]* levou a vitória por pontuar mais, já que, seguindo a formula para o motor de desempate o *jogador[0]* pontuou 2.819.809 pontos, enquanto o *jogador[2]* pontuou 2.819.098, uma diferença de 711 pontos, mesmo o *Jogador[2]* possuindo um kicker de Ás, porém os pares do *jogador[0]* eram superiores.

5.2 Evolução na taxa de exploração



Figura 2 – Decaimento do fator de exploração *epsilon* (ϵ).

O gráfico demonstra o decaimento do fator de exploração *epsilon* (ϵ), sendo possível notar que a partir de por volta do jogo 500, a taxa de exploração é mínima, significando que o agente está jogando com tudo o que aprendeu, ao invés de apenas tomar ações aleatórias.

5.3 Desempenho e lucro do agente

Durantes as simulações o agente a ser observado foi o responsável pelo *Jogador[0]* para facilitar o entendimento e plotação gráfica, é possível verificar o lucro ou prejuízo do agente observado no seguinte gráfico:

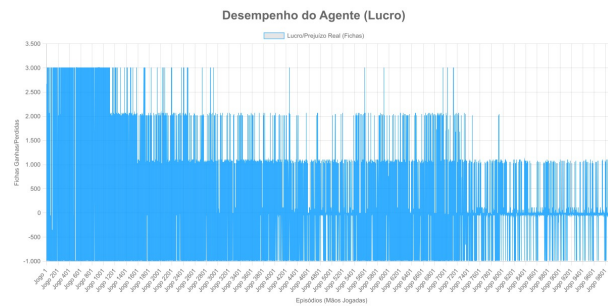


Figura 3 – Desempenho do agente ao longo das simulações

A variância praticamente infinita de estados do *Texas Hold'Em* gera um gráfico com grandes flutuações, porém é possível perceber que a flutuação de ganho e perda de fichas pelo agente cai drasticamente conforme as simulações, as primeiras simulações em que o agente está explorando e toma ações aleatórias a flutuação é maior que nas outras partes, que vão decaindo conforme a rede neural aprende a lidar melhor com os estados.

6 CONCLUSÕES E TRABALHOS FUTUROS

O principal objetivo deste trabalho era desenvolver um agente capaz de jogar o poquêr *Texas Hold'Em*, utilizando de uma rede neural e algoritmos de predição, sendo assim, o objetivo foi alcançado por meio de uma rede neural utilizando aprendizado por reforço e o algoritmo de predição *Monte Carlo*, o agente se mostrou coerente com o proposto, utilizando métodos de desempate robustos e complexos, enquanto eram evitados gargalos, aplicando o motor de pontuação de base 15, uma resolução matemática concreta e que se mostrou extremamente precisa na categorização de cada jogo possível do *Texas Hold'Em*.

Entretanto, pela natureza contínua e variável das apostas do *Texas Hold'Em*, o *Q-Learning* não viabiliza tal variação das apostas, já que tornaria os *Q-Values* ainda mais complexos de serem aplicados, sendo assim foi necessário fazer uma discretização das apostas, não dando liberdade para o agente escolher o tamanho da aposta, limitando o mesmo a apenas cinco ações, sendo elas: *Call/Check*, *Raise* leve, *Raise* forte, *fold* e *All-In*. Além de limitações performáticas matriciais, já que o *Node.js* não é tão eficiente quando se trata de cálculo de matrizes complexas. O programa também tem um ciclo de apostas fechado, não possibilitando um aumento cada vez maior das apostas, por exemplo, caso o último agente aposte uma quantia, o primeiro agente não é capaz de cobrir essa aposta, ou até mesmo aumentá-la.

Para trabalhos futuros pode-se sugerir a ideia de fazer a substituição do algoritmo de DQN por algoritmos capazes de lidar com ações contínuas de apostas, tornando o agente ainda mais próximo da realidade. Também pode ser citado fazer uma evolução do *Monte Carlo* permitindo um processamento feito pela *Graphics Processing Unit* (GPU), acelerando ainda mais o cálculo de *Equity*, além de permitir um aumento exponencial da quantidade de simulações. Também pode ser mencionado a possibilidade de haver um jogador humano jogando contra o agente inteligente, permitindo assim testar ainda mais o agente.

REFERÊNCIAS

- BARBOSA, A. H.; FREITAS, M. S. d. R.; NEVES, F. d. A. d. Confiabilidade estrutural utilizando o método de monte carlo e redes neurais. **REM: Revista Escola de Minas**, SciELO Brasil, v. 58, p. 247–255, 2005.
- BENGIO, Y. *et al.* **Deep learning**. [S.l.]: MIT press Cambridge, MA, USA, 2017. v. 1.
- BROECK, G. Van den; DRIESSENS, K.; RAMON, J. Monte-carlo tree search in poker using expected reward distributions. In: CALDERS, T.; TUYLS, K.; PECHENIZKIY, M. (Ed.). **Proceedings of the 21st Benelux Conference on Artificial Intelligence (BNAIC)**. Eindhoven, The Netherlands: [s.n.], 2009. Disponível em: <http://starai.cs.ucla.edu/papers/VdBBNAIC09.pdf>.
- BROWN, N.; SANDHOLM, T. Superhuman ai for multiplayer poker. **Science**, American Association for the Advancement of Science, v. 365, n. 6456, p. 885–890, 2019.
- CHEN, B.; ANKENMAN, J. **The mathematics of poker**. [S.l.]: ConJelCo LLC Pittsburgh, PA, 2006.
- GAMMA, E. *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software**. Reading, MA: Addison-Wesley, 1994. ISBN 978-0201633610.
- GIACOMO, P.; MARTIN, A. Validating a frequently used poker-playing heuristic with monte carlo simulation. In: **Proceedings of the 2015 Annual Meeting of the Western Decision Sciences Institute**. Maui, HI: [s.n.], 2015. Disponível em: http://wdsinet.org/Annual_Meetings/2015_Proceedings/papers/paper24.pdf. Acesso em: 15 fev. 2026.
- GRANGEIRO, P. H. S. **Agente de Poker baseado em Redes Neurais e Algoritmos de Monte Carlo**. GitHub, 2026. Disponível em: <https://github.com/PedroHercules0810/Agente-de-Poker-baseado-em-Redes-Neurais-e-Algoritmos-de-Monte-Carlo>.
- HANSSON, D. H. **Ruby on Rails: The MVC Architecture**. 2004. Documentação oficial: https://guides.rubyonrails.org/getting_started.html.
- HEINRICH, J.; SILVER, D. Deep reinforcement learning from self-play in imperfect-information games. **arXiv preprint arXiv:1603.01121**, 2016.
- KLEIJ, T. A. A. J. van der. **Monte Carlo Tree Search and Opponent Modeling through Sequences of Actions in Poker**. Dissertação (Mestrado) — University of Groningen, Groningen, The Netherlands, 2010. Disponível em: https://www.ai.rug.nl/~mwiering/Tom_van_der_Kleij_Thesis.pdf. Acesso em: 15 fev. 2026.
- LEONELLI, M. **Simulation and Modelling to Understand Change**. 2021. https://bookdown.org/manuele_leonelli/SimBook/. Acesso em: 15 fev. 2026.
- MNIH, V. *et al.* Human-level control through deep reinforcement learning. **nature**, Nature Publishing Group, v. 518, n. 7540, p. 529–533, 2015.
- MORAVČÍK, M. *et al.* Deepstack: Expert-level artificial intelligence in heads-up no-limit poker. **Science**, American Association for the Advancement of Science, v. 356, n. 6337, p. 508–513, 2017.

NG, A. Y.; HARADA, D.; RUSSELL, S. Policy invariance under reward transformations: Theory and application to reward shaping. In: CITESEER. **Icml**. [S.l.], 1999. v. 99, p. 278–287.

REENSKAUG, T. **Thing-Model-View-Editor: An Example from a Planning System User Interface**. [S.l.], 1979. Technical Note, disponível em: <https://www.cs.cmu.edu/~tong/history/mvc.html>.

RUBINSTEIN, R. Y.; KROESE, D. P. **Simulation and the Monte Carlo Method**. [S.l.]: John Wiley & Sons, 2008.

RUSSELL, S. J. **Artificial intelligence a modern approach**. [S.l.]: Pearson Education, Inc., 2010.

SMILKOV, D. *et al.* Tensorflow.js: Machine learning for the web and beyond. **Proceedings of machine learning and systems**, v. 1, p. 309–321, 2019.

SUTTON, R. S.; BARTO, A. G. **Reinforcement Learning: An Introduction**. 2. ed. Cambridge, MA: The MIT Press, 2018. ISBN 978-0262039246.

A SAÍDA DO CONSOLE

```
===== JOGO 20 =====

Usando a seed: 0.7739004342404441

--- Cobrança de Blinds ---
[BLIND] Jogador[2] pagou o Small Blind (25)
[BLIND] Jogador[3] pagou o Big Blind (50)

=== Fase: Pre-Flop ===

--- Turno do Jogador[0] ---
Cartas: [8 de Paus] e [8 de Ouros]
Fichas: 990 | Pote Atual: 115
[Parâmetros Ocultos da IA] -> Equity MC: 37.00% | Aposta a cobrir: 50
(Ação Aleatória - Explorando) - Ação escolhida: 1
Decisão: CALL/CHECK de 50

--- Turno do Jogador[1] ---
Cartas: [10 de Espadas] e [Ás de Espadas]
Fichas: 990 | Pote Atual: 165
[Parâmetros Ocultos da IA] -> Equity MC: 37.50% | Aposta a cobrir: 50
(Ação Aleatória - Explorando) - Ação escolhida: 1
Decisão: CALL/CHECK de 50

--- Turno do Jogador[2] ---
Cartas: [5 de Espadas] e [Ás de Copas]
Fichas: 965 | Pote Atual: 215
[Parâmetros Ocultos da IA] -> Equity MC: 27.50% | Aposta a cobrir: 25
(Ação Aleatória - Explorando) - Ação escolhida: 3
Decisão: RAISE FORTE de 325

--- Turno do Jogador[3] ---
Cartas: [6 de Copas] e [6 de Ouros]
Fichas: 940 | Pote Atual: 540
[Parâmetros Ocultos da IA] -> Equity MC: 29.00% | Aposta a cobrir:
    300
(Ação Aleatória - Explorando) - Ação escolhida: 4
```

```
Decisão: ALL-IN DE 940 FICHAS! (Aposta por valor)

=== Fase: Flop ===
Comunitárias na mesa:
3 de Paus
3 de Ouros
valete de Espadas

--- Turno do Jogador[0] ---
Cartas: [8 de Paus] e [8 de Ouros]
Fichas: 940 | Pote Atual: 1480
[Parâmetros Ocultos da IA] -> Equity MC: 38.00% | Aposta a cobrir: 0
(Ação Aleatória - Explorando) - Ação escolhida: 4
Decisão: ALL-IN DE 940 FICHAS! (Aposta por valor)

--- Turno do Jogador[1] ---
Cartas: [10 de Espadas] e [Ás de Espadas]
Fichas: 940 | Pote Atual: 2420
[Parâmetros Ocultos da IA] -> Equity MC: 27.00% | Aposta a cobrir:
    940
(Ação Aleatória - Explorando) - Ação escolhida: 3
Decisão: RAISE FORTE de 940

--- Turno do Jogador[2] ---
Cartas: [5 de Espadas] e [Ás de Copas]
Fichas: 640 | Pote Atual: 3360
[Parâmetros Ocultos da IA] -> Equity MC: 23.50% | Aposta a cobrir:
    940
(Ação Aleatória - Explorando) - Ação escolhida: 3
Decisão: RAISE FORTE de 640

=== Fase: Turn ===
Comunitárias na mesa:
3 de Paus
3 de Ouros
valete de Espadas
5 de Paus

=== Fase: River ===
```

```
Comunitárias na mesa:  
3 de Paus  
3 de Ouros  
valeta de Espadas  
5 de Paus  
valeta de Paus  
  
=> VENCEDOR DO SHOWDOWN: Jogador[0]  
Mão Final: Dois Pares (Score: 2819809)
```

Código-fonte 9 – Saída do console: Simulação do Jogo 20 - Rodadas e Showdown